



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2513 - TECNOLOGÍAS Y APLICACIONES WEB

Entrega 3

Tecnologías y Aplicaciones Web

2º semestre 2026 - Profesor Hernán Cabrera

Fecha de Entrega: Domingo 18 de Octubre a las 23:59 hrs.

Introducción

En la Entrega 1 diseñaron la arquitectura conceptual, el modelo relacional y las reglas de su juego, mientras que en la Entrega 2 dieron vida a la interfaz visual en React consumiendo datos simulados (*mock data*).

El objetivo central de esta **Entrega 3** es iniciar el desarrollo del **servidor Backend**, implementar la **persistencia de datos real** en una base de datos relacional y concretar la **primera integración real entre el cliente y el servidor**.

En esta etapa, el backend implementado sobre **Node.js** y el framework **Koa** asumirá el rol de servidor de la aplicación web, proveyendo una API RESTful estructurada bajo el patrón arquitectónico **MVC (Modelo-Vista-Controlador)**, con modelos gestionados mediante el ORM **Sequelize** sobre **PostgreSQL**. Asimismo, el Frontend de React (E_2) deberá sustituir sus datos mock por llamadas HTTP reales hacia los endpoints del servidor.

Indicaciones Generales

El proyecto deberá avanzar hacia una arquitectura cliente-servidor completamente funcional, cumpliendo con los siguientes requerimientos principales:

- Base de Datos y ORM:** Implementación del modelo Entidad-Relación en PostgreSQL usando Sequelize, incluyendo asociaciones y validaciones a nivel de modelo.
- Servidor Koa y Arquitectura MVC:** Estructuración modular y limpia del backend dividida en capas (*Routes, Controllers, Models, Services/Middlewares*).
- Operaciones CRUD Completas:** Endpoints funcionales para crear, consultar, actualizar y eliminar las entidades principales del juego, respetando el protocolo JSON acordado.
- Primera Integración Real Front-Back:** Reemplazo de los datos mock del frontend por peticiones HTTP (*fetch* o *axios*) hacia el servidor Koa.

- e. **Manejo de Errores y Códigos de Estado HTTP:** Respuestas semánticas adecuadas (200, 201, 400, 404, 500) ante solicitudes válidas e inválidas.
- f. **Documentación y Colección de Endpoints:** Exportación de la colección de pruebas en Postman o Swagger, junto con la bitácora de ejecución de jugadas.
- g. **Gestión del Proyecto:** Actualización y trazabilidad del Tablero Kanban y buenas prácticas en el repositorio (variables de entorno, ramas y PRs).

Requerimientos Detallados

1. Base de Datos Relacional y ORM (Sequelize + PostgreSQL)

A partir del diagrama Entidad-Relación definido en la Entrega 1 (y sus ajustes en E_2), deberán implementar el esquema de base de datos en **PostgreSQL** utilizando **Sequelize**:

- **Modelos y Migraciones:** Cada entidad relevante del juego (Usuarios, Partidas, Casillas/Tablero, Fichas/Recursos, etc.) debe contar con su modelo correspondiente y sus archivos de migración estructurados.
- **Asociaciones y Relaciones:** Definición explícita y correcta de relaciones (`hasOne`, `belongsTo`, `hasMany`, `belongsToMany`) con claves foráneas consistentes.
- **Validaciones en Modelos:** Reglas de integridad a nivel de ORM (campos no nulos, correos válidos, valores numéricos en rangos permitidos, unicidad de atributos, etc.).
- **Semillas (*Seeds*):** Scripts de población inicial (`seeders`) que permitan insertar datos de prueba reproducibles (usuarios base, tableros por defecto) para facilitar la evaluación.
- **Prohibición de Datos Hardcodeados:** Todo el estado persistente y los registros deben residir exclusivamente en la base de datos PostgreSQL.

2. Servidor Koa y Arquitectura Modular (MVC)

El backend debe desarrollarse sobre el entorno **Node.js** utilizando el framework **Koa**, siguiendo una separación de responsabilidades clara:

- **Enrutamiento (*Routes*):** Archivos de rutas modulares (usando `@koa/router`) que definan los caminos RESTful de la API de forma limpia.
- **Controladores (*Controllers*):** Encargados de procesar la solicitud (`ctx.request`), invocar la lógica de negocio y estructurar la respuesta (`ctx.body`).
- **Servicios / Lógica (*Services*):** Funciones auxiliares que aíslen reglas complejas y cálculos para evitar controladores sobrecargados.
- **Middlewares:** Uso apropiado de middlewares para el parseo del cuerpo de peticiones (`koa-body` o `koa-bodyparser`) y el manejo global de errores (CORS mediante `@koa/cors`).

3. Operaciones CRUD y Protocolo JSON

Se deben implementar operaciones **CRUD** completas (Create, Read, Update, Delete) para las entidades principales del sistema:

- **POST (Create):** Creación de nuevos registros (e.g., registro de usuarios, creación de nuevas partidas, asignación de elementos). Retornar código 201 **Created**.
- **GET (Read):** Consulta de recursos individuales y listados (e.g., obtener perfil de usuario, lista de partidas disponibles, estado de un tablero). Retornar código 200 **OK**.
- **PUT / PATCH (Update):** Actualización total o parcial de registros (e.g., modificar estado de una casilla, actualizar recursos de un jugador, cambio de turno).
- **DELETE (Delete):** Eliminación de recursos cuando corresponda (e.g., cancelar una partida en espera o eliminar un registro). Retornar 200 **OK** o 204 **No Content**.
- **Consistencia de Protocolo:** Los objetos JSON recibidos y entregados deben respetar el protocolo formal especificado en la Entrega 2 (cualquier ajuste debe quedar documentado en el `README.md`).

4. Primera Integración Real Front-Back

El Frontend en React desarrollado en la Entrega 2 debe conectarse directamente con la API de Koa:

- Reemplazar los archivos de datos mock locales por llamadas asíncronas reales (**fetch** o **axios**) hacia los endpoints del backend.
- Configurar **CORS** en el servidor Koa para permitir peticiones desde el cliente React.
- Las vistas deben reflejar en tiempo real la información proveniente de la base de datos (por ejemplo: la lista de partidas disponibles debe listarse desde PostgreSQL, y el envío de un formulario de registro debe guardar el nuevo usuario en la base de datos).

5. Manejo de Errores HTTP y Documentación de Endpoints

La API debe actuar de manera robusta ante situaciones anómalas:

- **Códigos de Estado HTTP Semánticos:** Respuestas explícitas con códigos 400 **Bad Request** (cuerpo malformado o campos faltantes), 404 **Not Found** (recurso inexistente) y 500 **Internal Server Error** (excepciones no controladas).
- **Colección de Pruebas (Postman / Swagger):** Se debe incluir en el repositorio el archivo `.json` exportado de la colección de **Postman** (o configuración de Swagger) con todos los endpoints creados, sus parámetros y ejemplos de cuerpo.
- **Simulación de Flujo en README:** Documentar el orden exacto de endpoints para probar el ciclo básico de juego (e.g., 1. Crear usuarios → 2. Crear partida → 3. Consultar tablero → 4. Realizar jugada/actualización).

6. Buenas Prácticas y Gestión del Proyecto

- **Variables de Entorno:** Configuración sensible (puerto, credenciales de PostgreSQL, URLs) debe residir en un archivo `.env`. **El archivo `.env` NO debe subirse al repositorio**; deben incluir un archivo `.env.example` con las variables requeridas.
- **README.md:** Instrucciones completas para configurar la base de datos, ejecutar migraciones/seeders y levantar tanto el backend como el frontend de manera local.
- **Tablero Kanban:** Registro activo y actualizado en GitHub Projects, Trello o Jira, evidenciando la distribución equitativa del trabajo y el estado de avance.

Condiciones y Restricciones

- **Stack Oficial:** Node.js, Koa, PostgreSQL, Sequelize y React. Prohibido el uso de TypeScript o lenguajes que compilen a JS.
- **Autenticación Segura (Nota):** En esta entrega la autenticación puede ser básica (envío y almacenamiento de datos para CRUD). El hashing con `bcrypt` y la emisión/protección estricta con **JWT** serán el foco principal de la Entrega 4.
- **Trabajo Colaborativo en GitHub:** Se evaluará el historial de commits y Pull Requests de todos los integrantes en los repositorios correspondientes.
- **No se aceptarán entregas por correo electrónico ni fuera del plazo oficial.**

Sobre la Calificación

La evaluación se basará en la correcta implementación del modelo relacional, la modularidad del servidor Koa, el funcionamiento de las operaciones CRUD y la primera integración exitosa con el cliente React.

Criterios de Evaluación

Criterio	Esencial	Intermedio	Completo	Pje.
1. Modelo de Datos y Sequelize	Modelos básicos sin migraciones	Modelos con migraciones pero relaciones incompletas	Esquema completo en PostgreSQL con Sequelize, migraciones, asociaciones y validaciones	15
2. Arquitectura MVC y Koa	Código concentrado en un archivo	Separación parcial de rutas y controladores	Arquitectura MVC modular estricta (routes, controllers, models, services), código limpio	15
3. Operaciones CRUD Completas	Solo métodos GET implementados	CRUD parcial o en una sola entidad	Operaciones CRUD completas (POST, GET, PUT/PATCH, DELETE) para las entidades del juego	15
4. Integración Front-Back Real	Frontend sigue usando datos mock	Integración en una sola vista	Conexión HTTP real entre React y Koa, reemplazo de mocks y consumo dinámico	15
5. Manejo de Errores y Códigos HTTP	Códigos genéricos (solo 200/500)	Manejo parcial de errores comunes	Uso semántico de códigos HTTP (200, 201, 400, 404, 500) y mensajes de error descriptivos	10
6. Colección de Endpoints (Postman)	Colección incompleta o con errores	Colección funcional pero sin ejemplos	Colección de Postman exportada (o Swagger) completa, con parámetros y payloads documentados	10
7. Simulación de Flujo y Seeders	No hay datos de prueba ni flujo claro	Flujo documentado pero sin datos iniciales	Seeders ejecutables y guía paso a paso en el README para simular el ciclo de juego	8
8. Consistencia de Protocolo y ER	Cambios drásticos no documentados	Modelo alineado pero con discrepancias	Modelo y endpoints 100 % consistentes con el diseño de E_1/E_2 y cambios documentados	4
9. Variables de Entorno y Buenas Prácticas	Archivo <code>.env</code> commiteado al repo	Variables dispersas en el código	Uso estricto de <code>.env</code> con <code>.env.example</code> , sin credenciales ni datos hardcoded	4
10. Gestión Ágil y Tablero Kanban	Tablero desactualizado o sin responsables	Tareas divididas por columnas	Tablero Kanban activo, asignación equitativa y trazabilidad clara del avance	4

Puntaje Total: 100 puntos

Requisitos Mínimos para Nota 4.0

Para optar a una calificación igual o superior a 4.0, el grupo debe cumplir obligatoriamente en nivel **Completo** con los siguientes ítems:

1. Modelo de Datos, migraciones, relaciones y validaciones en Sequelize.
2. Estructura modular del backend bajo el patrón MVC con Koa.
3. Operaciones CRUD completas para las entidades del sistema.
4. Integración HTTP real entre el cliente React y el servidor Koa (reemplazo de datos mock).
5. Manejo adecuado de errores HTTP y colección de endpoints exportada (Postman/Swagger).

Política de Referencia y Uso de Inteligencia Artificial

En el desarrollo del proyecto grupal, los estudiantes podrán utilizar material proporcionado por el equipo docente, recursos disponibles en internet y herramientas basadas en Inteligencia Artificial (por ejemplo, ChatGPT, Claude, GitHub Copilot, entre otras). Sin embargo, todas las fuentes utilizadas deben ser debidamente referenciadas y el equipo debe demostrar un **aporte propio significativo** en la solución implementada.

Si se utiliza Inteligencia Artificial, se deberá indicar de forma explícita, transparente y detallada en una sección dedicada del archivo `README.md` de la entrega (o en un documento anexo dentro de la carpeta `/docs`), incluyendo:

- **Alcance y contexto:** Indicar con precisión qué partes del trabajo, componentes, estilos o funciones fueron apoyadas o generadas mediante IA.
- **Registro de interacción:** Adjuntar los *prompts* utilizados, los fragmentos (*snippets*) de código entregados por la herramienta y los enlaces públicos a las conversaciones o **capturas de pantalla legibles de los chats**.
- **Aporte y análisis crítico:** Explicar qué modificaciones, adaptaciones, correcciones e integraciones realizó el equipo sobre dicho contenido para incorporarlo al proyecto.

Régimen de Penalizaciones y Descuentos:

- En caso de que el aporte de los integrantes no sea considerado sustancial (por ejemplo, si la mayor parte del trabajo corresponde a código generado automáticamente sin comprensión ni personalización por parte del grupo), la nota podrá ser rebajada según el criterio del equipo docente.
- **Uso no referenciado o referenciado incorrectamente:** El uso de herramientas de IA sin su debida declaración, o con un reporte incompleto (e.g., no adjuntar capturas, prompts o no transparentar el alcance real de su uso), constituye una falta grave a esta política y se aplicará un **descuento directo de entre 0 a 100 puntos sobre la calificación final de la entrega**, dependiendo de la magnitud y gravedad de la omisión detectada. El uso no declarado de IA podrá derivar adicionalmente en una calificación final de nota 1,1, además de las medidas disciplinarias que determine el equipo docente.